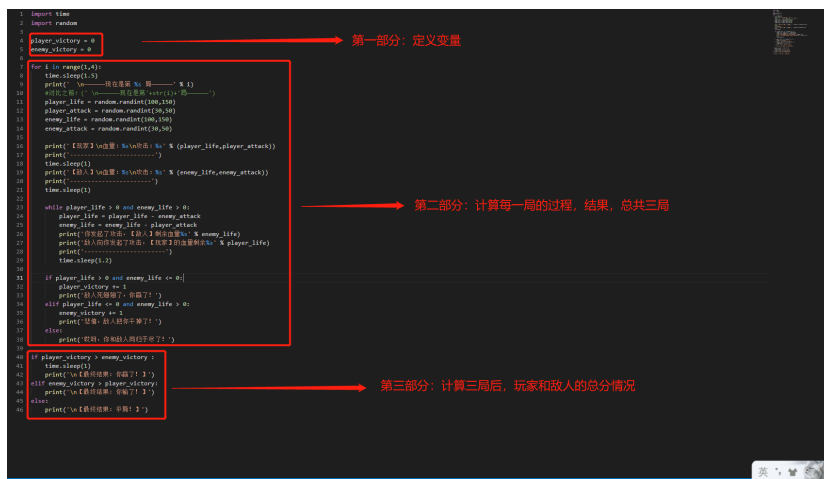


第7关 代码拆解与拓展

三局两胜 代码拆解



课后练习：代码拆解

#总体思路：

#1.1展示随机属性&随机生命值和攻击值

#1.2展示PK过程

#1.3展示每一局的PK结果，并设置三局两胜，就是在每一局外面设置for i in range(1,4)的循环

#1.4展示三局两胜的最后PK结果，借助第三方变量，类似篮球积分

#1.5设置三局两胜在输入某个键时，是否继续循环，也就是while True部分



拓展：格式化字符串

%填充

格式化字符串时，Python使用一个字符串作为模板。模板中有格式符，这些格式符为真实值预留位置，并说明真实数值应该呈现的格式。Python用一个**tuple**将多个值传递给模板，每个值对应一个格式符。

```
1 print("I'm %s. I'm %d year old" % ('Vamei', 99))
2 # 结果输出 I'm Vamei. I'm 99 year old
3 # 上面的例子中, "I'm %s. I'm %d year old" 为我们的模板。
4 # %s为第一个格式符, 表示一个字符串。
5 # %d为第二个格式符, 表示一个整数。
6 # ('Vamei', 99)的两个元素 'Vamei' 和99为替换%s和%d的真实值
```

在模板和tuple之间，有一个%号分隔，它代表了格式化操作。

整个“I'm %s. I'm %d year old” % ('Vamei', 99) 实际上构成一个字符串表达式。

我们可以像一个正常的字符串那样，将它赋值给某个变量。比如：

```
1 a = "I'm %s. I'm %d year old" % ('Vamei', 99)
2 print(a)
```

结果：

```
1 I'm Vamei. I'm 99 year old
```

我们还可以用词典来传递真实值。如下：

```
1 print("I'm %(name)s. I'm %(age)d year old" %
2 {'name': 'Vamei', 'age': 99})
```

结果：

```
1 I'm Vamei. I'm 99 year old
2 # 可以看到, 对两个格式符进行了命名。命名使用()括起来。每个命名对应词典的一个key。
```

format格式化填充

format是python2.6后新增的格式化字符串的方法，相对于老版的%格式方法，它有很多优点。

- 不需要理会数据类型的问题，在%方法中%s只能替代字符串类型
- 单个参数可以多次输出，参数顺序可以不相同
- 填充方式十分灵活，对齐方式十分强大
- 官方推荐用的方式，%方式将会在后面的版本被淘汰

1) 通过位置来填充字符串

```
1 print ('hello {0} i am {1}'.format('Kevin','Tom'))
2 # hello Kevin i am Tom
3 print ('hello {} i am {}'.format('Kevin','Tom'))
4 # hello Kevin i am Tom
5 print ('hello {0} i am {1} . my name is {0}'.format('Kevin','Tom'))
6 # hello Kevin i am Tom . my name is Kevin
```

format会把参数按位置顺序来填充到字符串中，第一个参数是0，然后1

也可以不输入数字，这样也会按顺序来填充

同一个参数可以填充多次，这个是format比%先进的地方。

2) 通过key来填充

```
1 print ('hello {name1} i am {name2}'.format(name1='Kevin',name2='Tom'))
2 # hello Kevin i am Tom
```

3) 通过下标填充

```
1 names = ['Kevin','Tom']
2 print('hello {names[0]} i am {names[1]}'.format(names=names))
3 # hello Kevin i am Tom
4 print ('hello {0[0]} i am {0[1]}'.format(names))
5 # hello Kevin i am Tom
```

4) 通过字典的key

```
1 names={'name':'Kevin','name2':'Tom'}
2 print ('hello {names[name]} i am {names[name2]}'.format(names=names))
3 # hello Kevin i am Tom
4 注意访问字典的key，不用引号。
```

5) 通过对象的属性

```
6 class Names():
7     name1='Kevin'
8     name2='Tom'
9     print ('hello {names.name1} i am {names.name2}'.format(names=Names))
10 # hello Kevin i am Tom
```